Part A - Dataset Understanding

Step 1: Environment Setup

git lfs install
huggingface-cli login (token: hf
qlVWyvDBWekOOvZKImhzktRWJDutIVtBsi) git clone
https://huggingface.co/datasets/A14A-lab/RecruitView

Step 2: Dataset Exploration

Write a report show your understanding on the dataset, you may include but are not limited to:

Q1

How many interview video clips are in the dataset?

2011 videos

[3]:
```
from pathlib import Path
from collections import Counter
import pandas as pd
import cv2
import json

video_dir = Path("/home/jovyan/Dataset/RecruitView(JW)/videos")
dataset_dir = Path("/home/jovyan/Dataset/RecruitView")

videos = sorted(video_dir.glob("*.mp4"))
print("Total video files:", len(videos))
```

```
Total video files: 2011
```

Q2

How many participants contributed to the dataset?

331 participants

•[19]:
```
possible_participant_cols = [
    col for col in df.columns
    if any(keyword in col.lower() for keyword in [
        "participant", "candidate", "user", "person", "subject", "interviewee", "session"
    ])
]

print("Possible participant columns:")
print(possible_participant_cols)
```

```
Possible participant columns:
['user_no', 'overall_personality']
```

•[20]:
```
if len(possible_participant_cols) > 0:
    participant_col = possible_participant_cols[0]
else:
    participant_col = None

print("Selected participant column:", participant_col)
```

```
Selected participant column: user_no
```

[21]:
```
if participant_col is not None:
    num_participants = df[participant_col].nunique(dropna=True)
    participants = sorted(df[participant_col].dropna().astype(str).unique())

    print("Number of participants:", num_participants)
    print("\nFirst 30 participants:")
    print(participants[:30])

    participant_summary = (
        df.groupby(participant_col)
        .size()
        .reset_index(name="number_of_videos")
        .sort_values("number_of_videos", ascending=False)
    )

    display(participant_summary)
else:
    print("No participant column found. Trying to extract participant ID from filename...")
```

```
Number of participants: 331

First 30 participants:
['1', '10', '100', '101', '102', '103', '104', '105', '106', '107', '108', '109', '11', '110', '111', '112', '113',
'114', '115', '116', '117', '118', '119', '12', '120', '121', '122', '123', '124', '125']
```

Q3

How many interview questions are available?

76 curated questions

[22]:
```
from pathlib import Path
import pandas as pd
import json

metadata_path = Path("/home/jovyan/Intern Assessment/RecruitView.json")

with open(metadata_path, "r", encoding="utf-8") as f:
    data = json.load(f)

if isinstance(data, list):
    df = pd.DataFrame(data)
else:
    df = pd.json_normalize(data)

print("Rows:", len(df))
print("Columns:")
print(df.columns.tolist())
```

```
Rows: 2011
Columns:
['id', 'video', 'duration', 'question_id', 'question', 'video_quality', 'user_no', 'openness', 'conscientiousness',
'extraversion', 'agreeableness', 'neuroticism', 'overall_personality', 'interview_score', 'answer_score', 'speaking
_skills', 'confidence_score', 'facial_expression', 'overall_performance', 'transcript', 'gemini_summary']
```

```
[23]: # Find possible question columns
question_cols = [
    col for col in df.columns
    if "question" in col.lower()
]

print("Possible question columns:", question_cols)
```

```
Possible question columns: ['question_id', 'question']
```

```
•[24]: question_col = question_cols[0]  # change if needed

num_questions = df[question_col].nunique(dropna=True)

print("There are", num_questions, "interview questions.")

df[[question_col]].drop_duplicates().reset_index(drop=True)
```

```
Q3. How many interview questions are available?
There are 76 interview questions.
```

Q4

What are the input modalities used by the system?

Videos, audios and text

```
•[25]: input_modalities = pd.DataFrame({
    "Modality": [
        "Video / Visual",
        "Audio / Speech",
        "Text / Language"
    ],
    "What it uses": [
        "Facial expression, posture, eye contact, body gestures",
        "Tone, pace, clarity, confidence, vocal delivery",
        "Transcript of the candidate's spoken answer"
    ]
})

display(input_modalities)
```

```
Q4. What are the input modalities used by the system?
```

	Modality	What it uses
0	Video / Visual	Facial expression, posture, eye contact, body ...
1	Audio / Speech	Tone, pace, clarity, confidence, vocal delivery
2	Text / Language	Transcript of the candidate's spoken answer

Q5

What are the output scores predicted by the model?

"Openness", "Conscientiousness", "Extraversion", "Agreeableness", "Neuroticism", "Overall Personality", "Interview Score", "Answer Score", "Speaking Skills", "Confidence Score", "Facial Expression", "Overall Performance"

```
[29]: output_scores_df = pd.DataFrame({
          "No.": range(1, len(output_scores) + 1),
          "Predicted Output Score": output_scores
      })

      print("The model predicts", len(output_scores), "continuous scores.")
      display(output_scores_df)
```

The model predicts 12 continuous scores.

	No.	Predicted Output Score
0	1	Openness
1	2	Conscientiousness
2	3	Extraversion
3	4	Agreeableness
4	5	Neuroticism
5	6	Overall Personality
6	7	Interview Score
7	8	Answer Score
8	9	Speaking Skills
9	10	Confidence Score

Q6 What is the main task of the dataset/model?

The main task is multimodal regression: using video, audio, and transcript information to predict personality traits and interview performance scores.

```
[35]: openness              float64
      conscientiousness     float64
      extraversion          float64
      agreeableness         float64
      neuroticism           float64
      overall_personality   float64
      interview_score       float64
      answer_score          float64
      speaking_skills       float64
      confidence_score      float64
      facial_expression     float64
      overall_performance   float64
      dtype: object
```

```
[36]: df[score_cols].describe()
```

[36]:

	openness	conscientiousness	extraversion	agreeableness	neuroticism	overall_personality	interview_score	answer_score	speaking_
count	2.011000e+03	2.011000e+03	2.011000e+03	2.011000e+03	2.011000e+03	2.011000e+03	2.011000e+03	2.011000e+03	2.011000
mean	-5.858129e-10	-1.810283e-09	-8.733420e-10	-2.707772e-09	5.822762e-10	-4.385845e-09	7.469935e-12	1.960056e-10	2.853436
std	1.126621e+00	8.769595e-01	1.098106e+00	1.200348e+00	4.868924e-01	1.203363e+00	1.230107e+00	1.175536e+00	1.277570
min	-7.799628e+00	-6.928705e+00	-7.184409e+00	-8.412302e+00	-2.576639e+00	-7.526180e+00	-7.860317e+00	-1.019905e+01	-9.406297
25%	-5.492694e-01	-4.483626e-01	-5.464184e-01	-5.656886e-01	-3.062534e-01	-6.114667e-01	-5.957018e-01	-6.105685e-01	-5.580228
50%	8.590045e-03	4.257322e-02	4.365189e-02	-1.596086e-02	2.560906e-02	-1.139338e-02	-2.744952e-02	6.046850e-03	1.987945
75%	5.222476e-01	4.851403e-01	5.439856e-01	5.477878e-01	3.034359e-01	6.169388e-01	5.454144e-01	5.787526e-01	6.158908
max	9.338849e+00	6.617010e+00	8.910198e+00	9.242025e+00	2.225906e+00	9.269542e+00	9.386007e+00	8.722344e+00	7.901472

[32]:

```
output_scores = [
    "Openness",
    "Conscientiousness",
    "Extraversion",
    "Agreeableness",
    "Neuroticism",
    "Overall Personality",
    "Interview Score",
    "Answer Score",
    "Speaking Skills",
    "Confidence Score",
    "Facial Expression",
    "Overall Performance"
]

print("Number of output scores:", len(output_scores))
print("Because these are numerical scores, the task is regression.")
```

```
Number of output scores: 12
Because these are numerical scores, the task is regression.
```

Issue Type
Affected Video
Description
Suggested Data Cleaning method

Summary (number of video be4 cleaning vs number of video after cleaning)

Part C: dataset annotation

You are provided with 9 interview videos.
- recordings

Review all 9 videos and annotate at least 3 videos using your proposed annotation schema.
You may design what label should be included based on the candidate (Body gesture, Facial Expression, Speaking tone, and Content delivered by the candidate)

The final report outcome:

1. Annotation workflow (flowchart / any method)
2. Annotation Schema design, eg.

```
"video id": "001",
"candidate id": "candidate 01",
"annotations": (
        "confidence": 4,
        "speaking skill": 3,
        "facial_e pression": 5,
        "body gesture": 4,
        "answer_quality": 4
```

3. .json file to store your annotation
4. Justify why each label is important for interview assessment

Part D - AI Interview Pipeline Design

Design an end-to-end AI Interview Assessment System.
References (you may find additional references beyond these):

1. RecruitView Paper
   https://arxiv.org/pdf/2512.00450
2. Persuasiv AI
   https://persuasiv.ai/en
3. Internal Demo System
   https://lilac-b1cmish-ongoing.ngrok-frcc.dcv/

Task

1. Study the references and propose your own Al Interview Assessment Pipeline.

2. You are NOT required to replicate the existing systems exactly.

3. You may improve, simplify, or redesign the workflow based on your understanding.

In the report, include:

1. Provide a complete flow diagram., eg.

   Video Input

   Video

   Processing

   Audio

   Processing

   Transcript Generation

   Feature Extraction

   Model

   Final Assessment Scores

   *You may include additional modules if necessary.

2. Explanation of each component in the pipeline
3. Compare your proposed system against the provided references:
   a. RecruitView
   b. Internal Demo System
   c. Other reference found
4. Design an evaluation process to determine whether the AI Interview model is performing well.

   Include:
   Evaluation
   Metrics
   Validation
   Strategy Testing
   Procedure
   Human Evaluation Procedure
   Model Comparison
   Methodology